

Technology Stack

Date	29 June 2026
Team ID	N/A
Project Name	OptiCrop Smart Agricultural Production Optimization Engine
Maximum Marks	2 Marks

Define Your Technology Stack: Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, Bootstrap 5, CSS	Provides a responsive, browser-based user interface for the Home, About, and Find Your Crop pages. Bootstrap 5 ensures clean layout and mobile compatibility without additional tooling.
2	Backend / Server-Side	Python 3.x, Flask	Flask serves as the lightweight WSGI web framework that routes HTTP requests, handles form submissions (POST), loads the trained model at startup, and renders Jinja2 HTML templates with prediction results.
3	Machine Learning Layer	Scikit-learn (Random Forest, Logistic Regression, KNN, Decision Tree, K-Means) NumPy, Pandas, SciPy	Scikit-learn provides all classification and clustering algorithms. NumPy and Pandas handle data loading and numerical operations. SciPy supports statistical computation during analysis and preprocessing.
4	Data Visualization	Matplotlib, Seaborn	Matplotlib generates all static plots (distribution histograms, elbow curve, confusion matrix). Seaborn provides high-level statistical charts (heatmaps, boxplots, scatter plots, pairplots) used in EDA.
5	Model Persistence	Pickle (Python standard library)	Serializes the trained Random Forest classifier (model.pkl), StandardScaler (scaler.pkl), and LabelEncoder (label_encoder.pkl) to disk so the Flask app can load them at startup without retraining.
6	Database / Data Storage	N/A — Not Used	The application does not use a persistent database. User inputs are processed in memory per request. The training dataset (Crop_recommendation.csv) is a static

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
7	Hosting / Deployment	Local — Flask Development Server (http://127.0.0.1:5000)	CSV file used only during model training. The application runs locally using Flask's built-in development server. No cloud deployment is required for this submission. Cloud deployment (Render, Railway) is noted as a future scope enhancement.
8	Development Environment	VS Code Jupyter Notebook Python venv	VS Code is the primary IDE for editing app.py and HTML templates. Jupyter Notebook (via VS Code's Jupyter extension) is used for EDA, preprocessing, and model training. Python venv isolates project dependencies.
9	Version Control & CI/CD	Git & GitHub	Git manages local source code history with meaningful commit messages. GitHub hosts the public repository (github.com/neelimavana/OptiCrop) for mentor review, submission, and collaboration.
10	Dataset / External Data Source	Crop Recommendation Dataset (Kaggle, CC0)	A publicly available agricultural dataset with 2,200 samples, 22 balanced crop classes (100 samples each), and 7 features: N, P, K, temperature, humidity, pH, rainfall. Downloaded from Kaggle as Crop_recommendation.csv.